

Lab #1: Discover and Connect to Data in OneLake

Certification Track: DP600 — Fabric Analytics Engineer Associate

Purpose

This lab walks us through how to discover and connect to data stored in OneLake by using the catalog, workspace shortcuts, Delta Lake tables, SQL endpoints, and semantic models. It introduces the core techniques used to locate data assets, understand their structure, and access them across workspaces without duplication. These steps establish the foundational workflows that data engineers rely on when working with OneLake in real analytics projects.

Key Skills Demonstrated

This lab develops foundational skills required for working with Microsoft Fabric and OneLake in real analytics environments. The following capabilities were demonstrated:

- Creating and managing Fabric workspaces
- Building Lakehouses and organizing data in OneLake
- Importing raw files and converting them into Delta Lake tables
- Discovering and understanding data assets through the OneLake catalog
- Creating shortcuts for cross-workspace access without data duplication
- Querying Delta Lake tables using the SQL analytics endpoint
- Aggregating and analyzing data using SQL
- Building semantic models with Direct Lake mode
- Exploring semantic model fields and measures
- Visualizing data using Fabric's analytics preview
- Cleaning up resources to maintain capacity efficiency

These skills form the core of the DP-600 Fabric Analytics Engineer workflow and establish a strong foundation for more advanced labs.

Tools Used

This lab utilized several components of the Microsoft Fabric ecosystem:

- **Microsoft Fabric Workspace** — Central environment for organizing all lab assets
- **Lakehouse** — Unified storage and analytics layer built on Delta Lake
- **OneLake Catalog** — Centralized discovery and metadata system
- **Shortcuts** — Cross-workspace links enabling single-copy architecture
- **Delta Lake Tables** — ACID-compliant storage format for structured analytics

- **SQL Analytics Endpoint** — Relational query interface for Lakehouse tables
- **Semantic Model** — Governed business layer for reporting and metrics
- **Analytics Preview (Power BI)** — Visualization interface for semantic models

These tools work together to provide a seamless, end-to-end analytics workflow.

What I Learned

This lab reinforced how Microsoft Fabric unifies storage, SQL analytics, and semantic modeling into a single, governed environment. I learned how Lakehouses store data in Delta Lake format, how the OneLake catalog centralizes discovery, and how shortcuts enable cross-workspace access without duplication. Querying data through the SQL analytics endpoint demonstrated how Fabric exposes Delta Lake tables through a relational interface, supporting familiar SQL workflows.

Building a semantic model highlighted the importance of standardized business logic, reusable measures, and governed relationships. The analytics preview showed how these models support consistent reporting across tools and users. Overall, the lab demonstrated how Fabric simplifies end-to-end analytics by integrating ingestion, discovery, querying, and modeling into a unified platform.

1. Create Workspace

Purpose: Establish the controlled environment where all lab items will be created, stored, and governed.

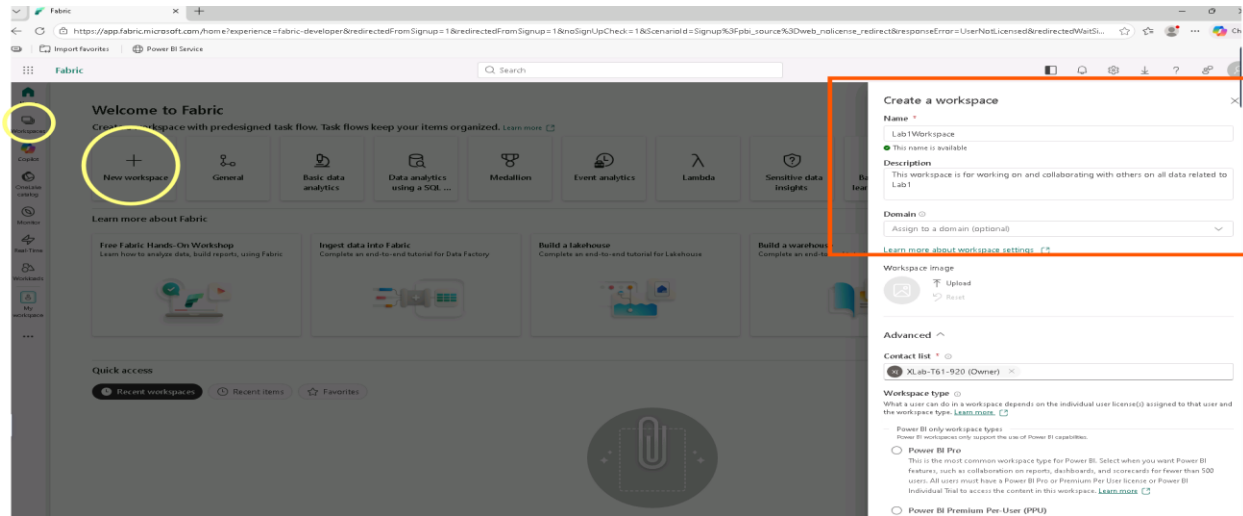
A workspace is a controlled environment used to store, organize, and manage Fabric items such as Lakehouses, warehouses, notebooks, pipelines, and semantic models. It defines the security boundary, access permissions, and collaboration scope for a project, ensuring that all data assets and development artifacts remain grouped, governed, and accessible within the same location.

Steps completed:

- Select **Workspaces**
- Choose **+ New workspace**
- Enter the workspace name – (*Lab1Workspace*)
- Select **Create**

A new workspace is generated and is displayed as empty and ready for development.

Figure 1 — Creating the Lab1Workspace workspace



The yellow circles indicate two entry points for creating a new workspace. Both options lead to the same creation screen. The orange box highlights where the workspace name and description are entered.

Structure:

- Workspaces act as containers for Fabric assets.
- They provide separation, organization, and governance for project-specific resources.

Function:

- Establishes the environment where all subsequent lab/project items will be created and managed.

Summary:

This step creates the foundational workspace that will host all Lakehouses, tables, shortcuts, and SQL queries used throughout the lab.

With the workspace created, the next step is to establish a Lakehouse that will store and expose the lab's data.

2. Create Lakehouse

Purpose: Create the primary storage and analytics environment for the lab.

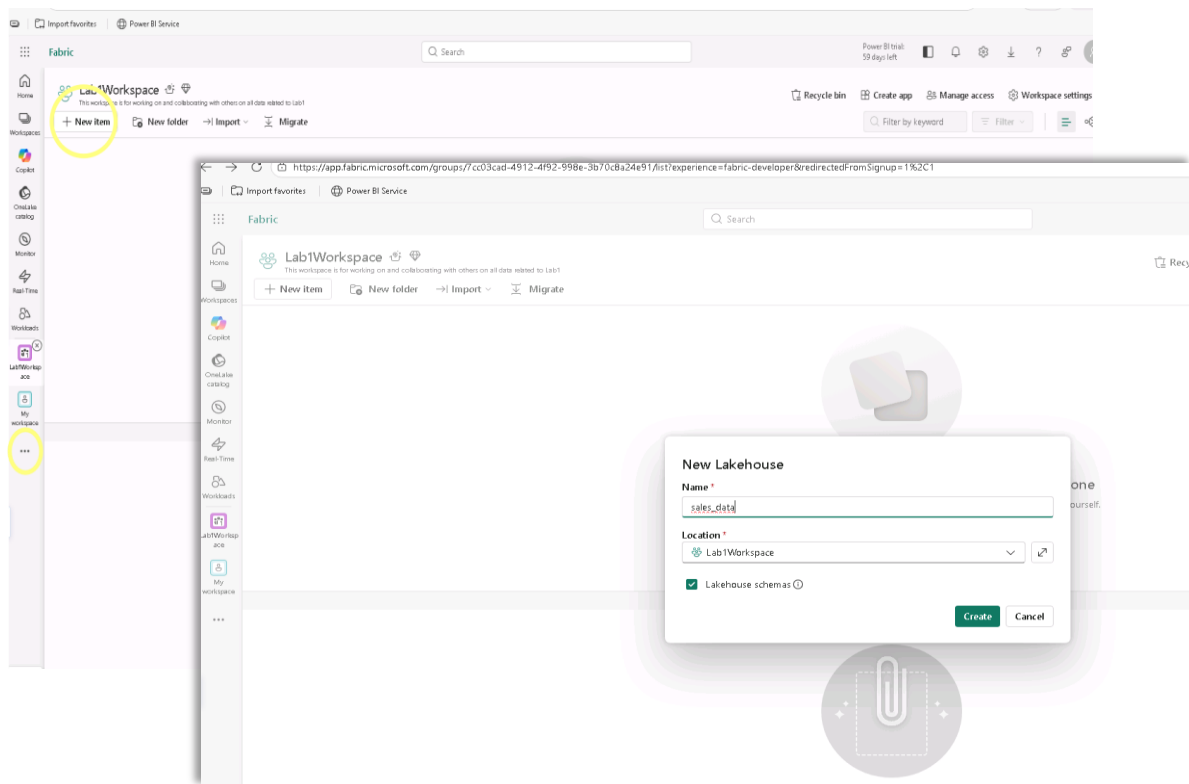
A Lakehouse provides a unified storage and analytics environment built on Delta Lake. It stores structured and unstructured data in OneLake and exposes that data through both file-based and table-based interfaces.

Steps completed:

- Select **+ New item**
- Choose **Lakehouse**
- Enter the Lakehouse name (**sales_data**)
- Select **Create**

A new Lakehouse is generated and displayed with default folders for Files and Tables.

Figure 2 & 3 — Creating the `sales_data` Lakehouse



The background image shows two entry points for creating a Lakehouse, indicated by yellow circles. Both options lead to the creation screen shown in the foreground image.

Structure:

- A Lakehouse stores data in OneLake using Delta Lake format.
- It provides both a file system view and a table view for analytics workflows.
- Lakehouse's serve as foundational storage for ingestion, transformation, and modeling.

Function:

- Establishes the primary data storage location for the lab.
- Provides the Delta Lake tables used in subsequent discovery, shortcut, SQL, and semantic model exercises.

Summary:

The Lakehouse now serves as the central location for all data used in the lab.

With the Lakehouse created, the next step is to import raw data into it for analysis.

3. Import Data into Lakehouse

Purpose: Load raw CSV data into the Lakehouse to prepare it for transformation and analysis.

The data used in this lab is sourced from Microsoft's public GitHub repository. The file is a CSV containing sales records that will be uploaded into the **sales_data** Lakehouse for further exploration and transformation.

GitHub source: <https://raw.githubusercontent.com/MicrosoftLearning/dp-data/main/sales.csv>

3.1 Retrieve Source Data from GitHub

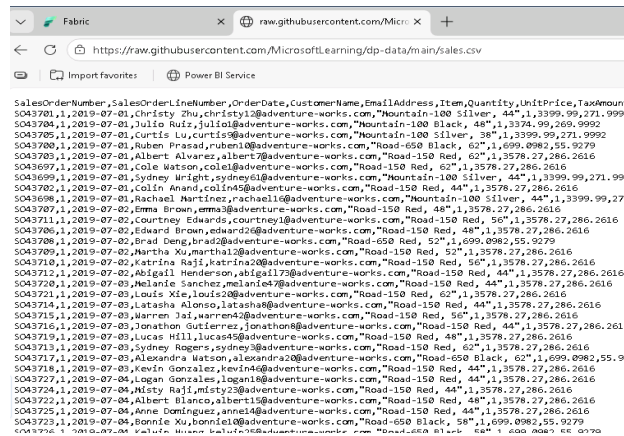
Fabric supports many ingestion methods, including uploading local files, connecting cloud storage, creating shortcuts, using pipelines, or ingesting data through notebooks. For the purposes of this lab, the dataset will be loaded from an external source in the form of a CSV file hosted in GitHub.

Steps completed:

- Select the GitHub link to open the CSV file
- Save the CSV file to the desktop for use in Fabric.

The CSV file is opened in a separate browser window and saved locally so it can be uploaded into the Lakehouse.

Figure 3 — Viewing and saving sales.csv from GitHub



This image shows the hosted CSV file opened in GitHub and prepared for download to the desktop.

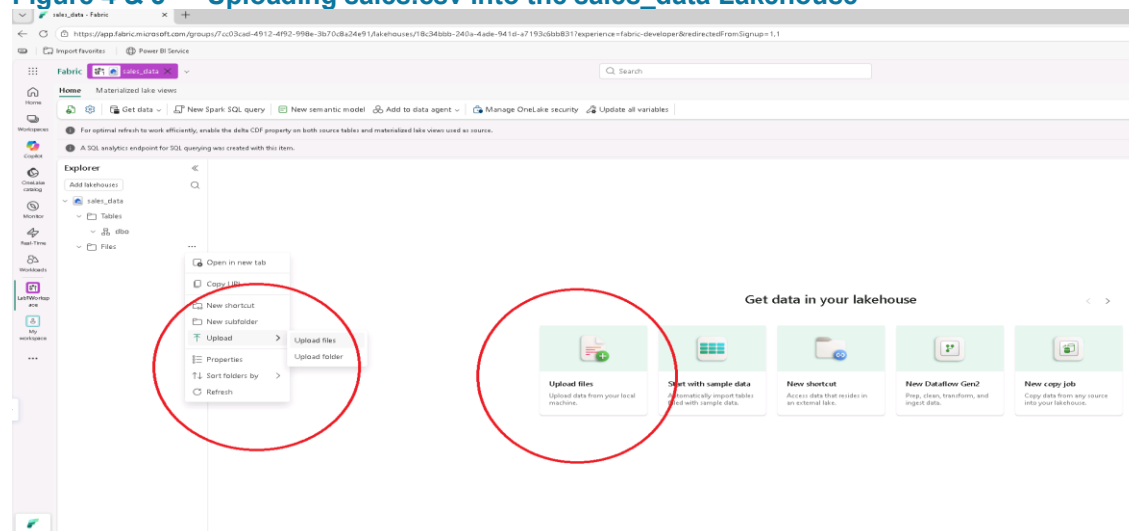
3.2 Load the File into Fabric

Steps completed:

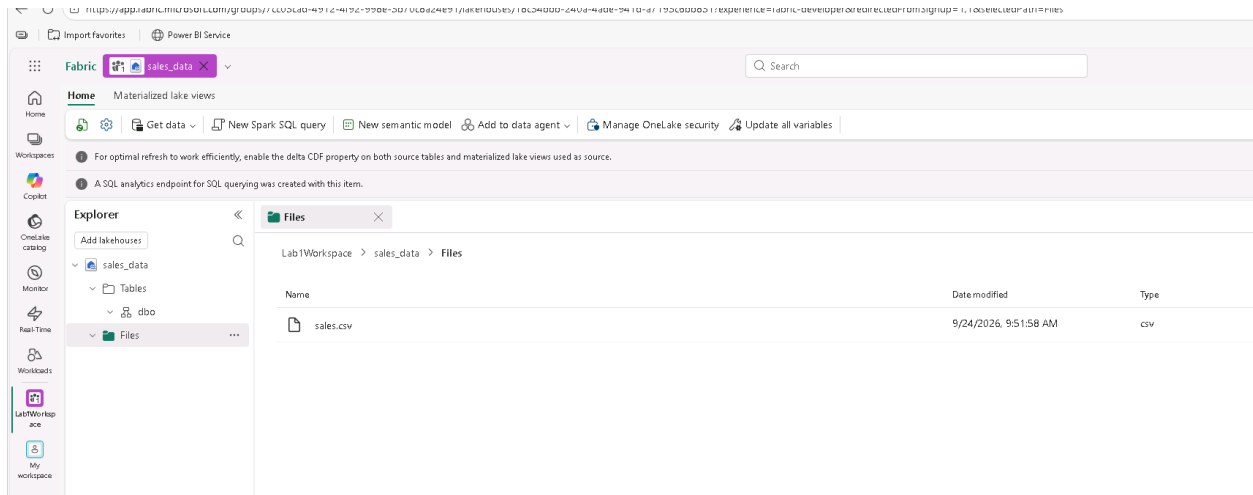
- Open the **sales_data** Lakehouse
- Select **Files**
- Choose **Upload**
- Select **Upload from local file**
- Browse to the saved CSV file Select **Upload**

The CSV file is added to the Lakehouse and appears under the **Files** section.

Figure 4 & 5 — Uploading sales.csv into the sales_data Lakehouse



The Fabric upload interface with two available options for adding data from a local file into the Lakehouse: The Upload command in the **Files** menu and the **Upload files** tile in the Get data panel.



The **sales.csv** file successfully uploaded and visible under *Lab1Workspace* → *sales_data* → *Files*.

Structure:

- The imported CSV file becomes part of the Lakehouse's file system.
- Files stored in the Lakehouse can be converted into Delta Lake tables for analytics workflows.
- GitHub acts as an external data source, providing raw input for ingestion.

Function:

- Supplies the dataset used to create Delta Lake tables in subsequent steps.
- Establishes the raw data foundation for catalog discovery, SQL queries, shortcuts, and semantic model creation.

Summary:

The Lakehouse now serves as the central location for all data used in the lab.

With the Lakehouse created, the next step is to import raw data into it for analysis.

4. Create Table from File

Purpose: Convert raw file data into a structured Delta Lake table that supports SQL queries and catalog discovery.

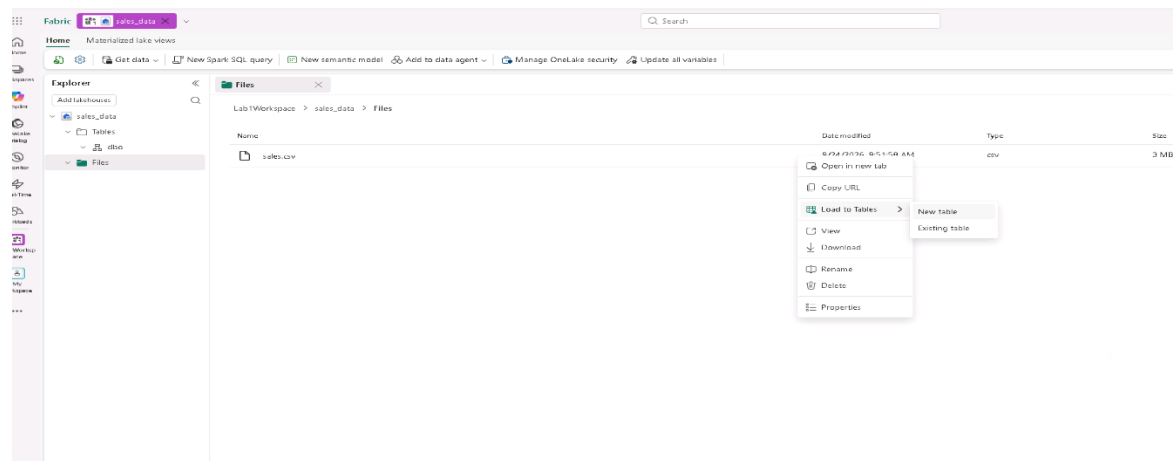
The uploaded CSV file can now be converted into a Delta Lake table within the Lakehouse. Delta Lake enhances standard Parquet storage by adding a transaction log that tracks every change made to the table. This allows the table to support reliable ACID operations, time-travel queries, and safe concurrent updates—capabilities not available in plain Parquet folders. Converting the CSV file into a Delta Lake table prepares it for SQL querying, catalog discovery, and downstream analytics.

Steps completed:

- Right-click the **sales.csv** file in the *Files* section
- Select **Load to Tables** → **New table**
- Confirm the table name (**sales**)
- Select **Load**

Fabric automatically creates a Delta Lake table and stores it under *Tables* → *dbo* in the Lakehouse.

Figure 6 — Creating a Delta Lake table from sales.csv



This image shows the **Load to Tables** → **New table** command used to convert the uploaded CSV file into a Delta Lake table.

Structure:

- The new table is stored under **Tables** → **dbo** in the Lakehouse.
- Each column is inferred from the CSV header and typed automatically.
- The table is query-ready through the Lakehouse SQL endpoint.
- Delta Lake stores the table as Parquet files and maintains an ordered transaction log in the **_delta_log** folder.
- Parquet provides efficient columnar storage, compression, and fast analytical reads. The transaction log records every change to the table as a sequential JSON commit file, and periodic checkpoint files improve read performance.

- **ACID** reliability means the table supports *Atomicity* (operations complete fully or not at all), *Consistency* (data always follows defined rules), *Isolation* (concurrent operations do not interfere), and *Durability* (committed changes are permanently preserved). These guarantees come from the transaction log, not the Parquet files themselves.

Function:

- Converts raw file data into structured, queryable Delta Lake format.
- Ensures reliable reads and writes through ACID-compliant transaction logging.
- Enables downstream analytics, catalog registration, and semantic modeling.

Summary

The CSV file has been transformed into a Delta Lake table, enabling SQL queries and catalog discovery.

With the Delta Lake table created, the next step is to locate it through the OneLake catalog.

5. Discover Data in Catalog

Purpose: Use the OneLake catalog to locate and understand data assets across workspaces.

The OneLake catalog provides a unified discovery experience for all data assets stored across workspaces. It allows analytics engineers to locate tables, files, warehouses, Lakehouses, and semantic models without navigating individual workspaces manually. Once the Delta Lake table is created, it becomes visible in the catalog with metadata describing its structure, schema, and storage location.

Using the catalog ensures consistent data governance, simplifies cross-workspace access, and provides a searchable inventory of all registered assets.

5.1 Search for the Lakehouse Table

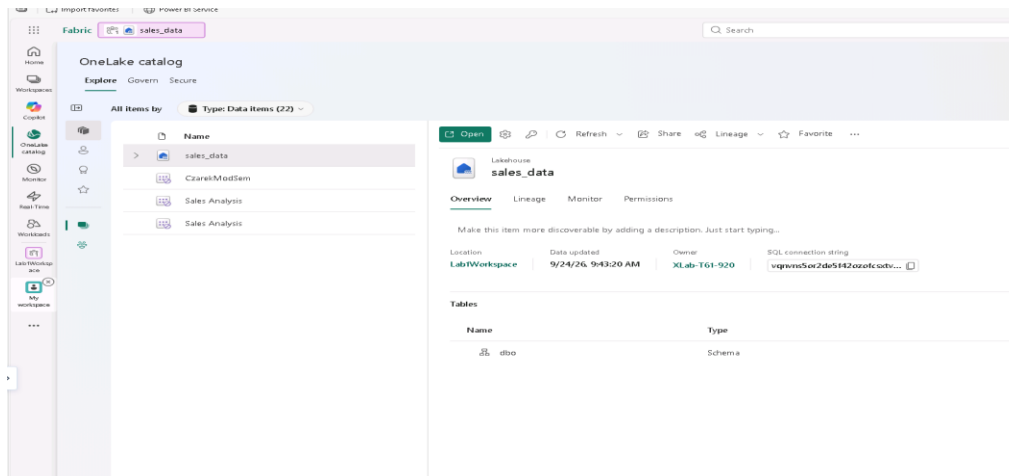
Steps completed:

- Open the **OneLake Data Hub**
- Select **Browse**
- Search for **sales_data**
- Open the Lakehouse entry

- Locate the **sales** table under the Lakehouse's registered assets

The catalog displays the Lakehouse and its associated tables, including the newly created **sales** Delta Lake table. Metadata such as table name, schema, storage location, and item type are shown in the catalog entry.

Figure 7 - Discovering the sales_data Lakehouse and its registered sales table in the OneLake catalog



This image shows the *sales_data* Lakehouse entry in the OneLake catalog. The overview panel lists metadata including workspace location (**Lab1Workspace**), last update time, owner, SQL connection string, and the **dbo** schema under Tables. This confirms that the Lakehouse and its Delta Lake table are successfully registered and discoverable through the catalog.

5.2 View Table Metadata

Steps completed:

- Select the **sales** table
- Open the **Details** pane
- Review metadata fields such as:
 - Table name
 - Item type (Delta Lake table)
 - Storage location in OneLake
 - Schema (dbo)
 - Workspace (Lab1Workspace)

The catalog provides a consistent metadata view that helps engineers understand the table's structure and lineage without opening the Lakehouse directly.

Structure:

- The OneLake catalog indexes all data assets across workspaces.
- Each table entry includes metadata such as schema, storage path, item type, and workspace.
- Delta Lake tables appear as catalog items once created, enabling cross-workspace discovery.

Function:

- Provides a centralized location to find and understand data assets.
- Enables engineers to locate tables without navigating workspace folders.
- Supports governance, lineage, and metadata-driven analytics workflows.
- Establishes the foundation for creating shortcuts, querying via SQL endpoints, and building semantic models.

Summary:

The catalog confirms that the Delta Lake table is registered, discoverable, and ready for cross-workspace access.

With the table discoverable, the next step is to access it from another Lakehouse using a shortcut.

6. Create Shortcut to Lakehouse Data

Purpose: Enable cross-workspace access to Delta Lake tables without duplicating data.

Shortcuts in OneLake allow data engineers to reference data stored in other workspaces or Lakehouses without copying it. They act as symbolic links that point to existing data locations, enabling unified access across projects while preserving a single source of truth. Creating a shortcut to the **sales_data** Lakehouse demonstrates how Fabric connects datasets between workspaces for collaborative analytics.

6.1 Create a Table Shortcut

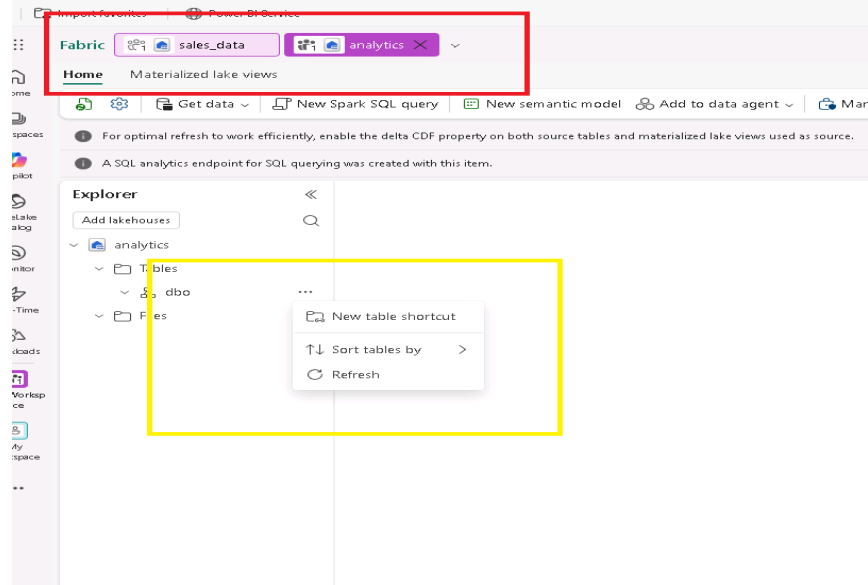
Steps completed:

- Open the **analytics** Lakehouse
- Select **Tables**
- Choose **New table shortcut**
- Specify the source Lakehouse (**sales_data**) and target table (**sales**)
- Enter the shortcut name (**sales**)

- Select **Create**

Fabric creates a logical link to the external table. The shortcut appears under *Tables* → *dbo* in the analytics Lakehouse, allowing direct access to the referenced data without duplication.

Figure 9 — Creating a table shortcut from analytics to the sales_data Lakehouse



This image shows the **analytics** Lakehouse interface with the **new table shortcut** option highlighted. The red box identifies both open Lakehouses with the active Lakehouse highlighted (*sales_data* and *analytics*), while the yellow box highlights the shortcut creation menu. This step establishes a cross-workspace link to the **sales** table stored in *sales_data* Lakehouse.

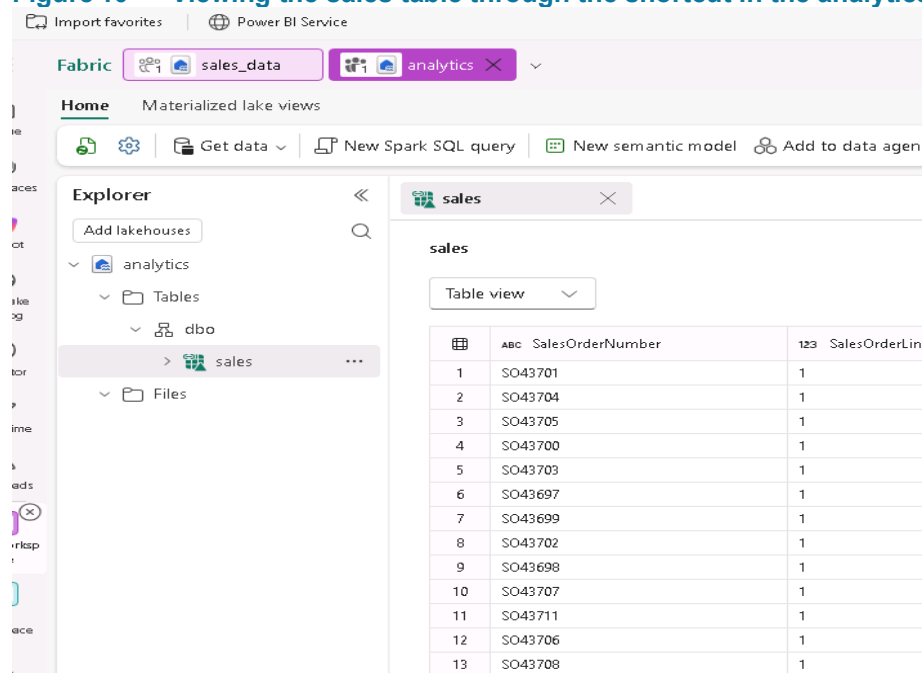
6.2 Verify Shortcut Access

Steps completed:

- Expand the *Tables* section in the **analytics** Lakehouse
- Locate the **sales** shortcut under *dbo*
- Open the shortcut to confirm that the external data is accessible
- Review the table contents to verify successful linkage

Fabric validates the shortcut path and displays the linked table as if it were stored locally in the analytics Lakehouse.

Figure 10 — Viewing the sales table through the shortcut in the analytics Lakehouse



This image shows the **sales** table opened via the shortcut. The table view confirms that the data from *sales_data* is accessible directly within *analytics*, displaying columns such as **SalesOrderNumber** and **SalesOrderLineNumber**.

Structure:

- A shortcut is a metadata link that references an existing data location.
- No data is copied; the shortcut points directly to the source table.
- Avoiding data copies reduces storage costs, eliminates synchronization issues, and ensures all users work from the same authoritative dataset.
- Shortcuts maintain OneLake's single-copy architecture, ensuring consistency across workspaces.
- They reference the same underlying Delta Lake files and transaction log, inheriting ACID guarantees from the source table.
- Shortcuts can reference other Lakehouses, dataflows, or external storage locations.

Function:

- Enables cross-workspace data access without duplication.
- Improves efficiency by preventing redundant data movement and keeping updates instantly visible across all linked workspaces.
- Ensures consistent analytics by allowing all workspaces to read the same authoritative Delta Lake table.
- Supports governed data sharing by maintaining source-controlled permissions and avoiding unmanaged copies.

- Simplifies multi-workspace analytics design by exposing shared datasets through logical links.
- Forms the foundation for unified analytics across multiple Fabric items.

Summary:

The shortcut provides seamless access to the Delta Lake table without copying or moving data.

With cross-workspace access established, the next step is to query the data using the SQL endpoint.

7. Query Lakehouse Data Using SQL Endpoint

Purpose: Use SQL to analyze Delta Lake data directly within the Lakehouse.

The SQL analytics endpoint provides a direct query interface to Delta Lake tables stored in OneLake. It allows engineers to use standard SQL syntax to analyze data without moving or transforming it. Queries run against the same underlying Delta Lake storage, ensuring consistency with all other Fabric items such as notebooks, pipelines, and semantic models.

Using the SQL endpoint demonstrates how Fabric integrates data engineering and analytics workflows within a single environment.

7.1 Run Item-Level Aggregation Query

Steps completed:

- Open the **analytics** Lakehouse
- Select **New SQL query**
- Enter the following query:


```
SELECT
    Item,
    SUM(Quantity * UnitPrice) AS TotalRevenue,
    SUM(Quantity) AS TotalQuantity
From sales
GROUP BY Item
ORDER BY TotalRevenue DESC;
```
- Run the query to calculate total revenue and total quantity per item.

Figure 11 — Querying item-level sales data using the SQL endpoint

The screenshot shows the Microsoft Fabric SQL query editor. The query is as follows:

```

1 SELECT
2   Item,
3   SUM(Quantity * UnitPrice) AS TotalRevenue,
4   SUM(Quantity) AS TotalQuantity
5 FROM sales
6 GROUP BY Item
7 ORDER BY TotalRevenue DESC;

```

The results panel displays the following data:

Item	TotalRevenue	TotalQuantity
Road-150 Red, 40	1205076.99	337
Road-150 Red, 62	1202296.72	336
Road-150 Red, 52	1080071.54	302
Road-150 Red, 36	1055088.65	295
Road-150 Red, 44	1005493.87	281
Mountain-200 Black, 46	925846.400100000	425
Mountain-200 Black, 42	844474.353300000	388
Mountain-200 Silver, 46	816222.180000000	371
Mountain-200 Silver, 38	815145.850000000	370
Mountain-200 Black, 38	799140.301100000	366
Mountain-200 Silver, 42	791116.592700000	359

This image shows the SQL query editor and results panel. The query aggregates total revenue and quantity by item, ranking results in descending order of revenue. The output confirms that the **sales** table is fully accessible through the SQL endpoint and supports analytical queries directly within Fabric.

Step-by-Step Breakdown

- **FROM sales:** The database pulls the raw data from the **sales** table.
- **GROUP BY Item:** The query bundles the raw sales rows together so that calculations happen per unique product.
- **SUM(Quantity * UnitPrice) AS TotalRevenue:** It multiplies the amount sold by the price per unit for each transaction, adds them up for each item, and labels the result column as **TotalRevenue**.
- **SUM(Quantity) AS TotalQuantity:** It counts the total number of units sold for each item and labels it **TotalQuantity**.
- **SELECT Item, ...:** It displays the product name alongside the calculated revenue and quantity totals.
- **ORDER BY TotalRevenue DESC:** It sorts the final list so the top-earning items appear at the very top.

7.2 Run Customer-Level Aggregation Query

Steps completed:

- Create a new query tab Enter the following query:

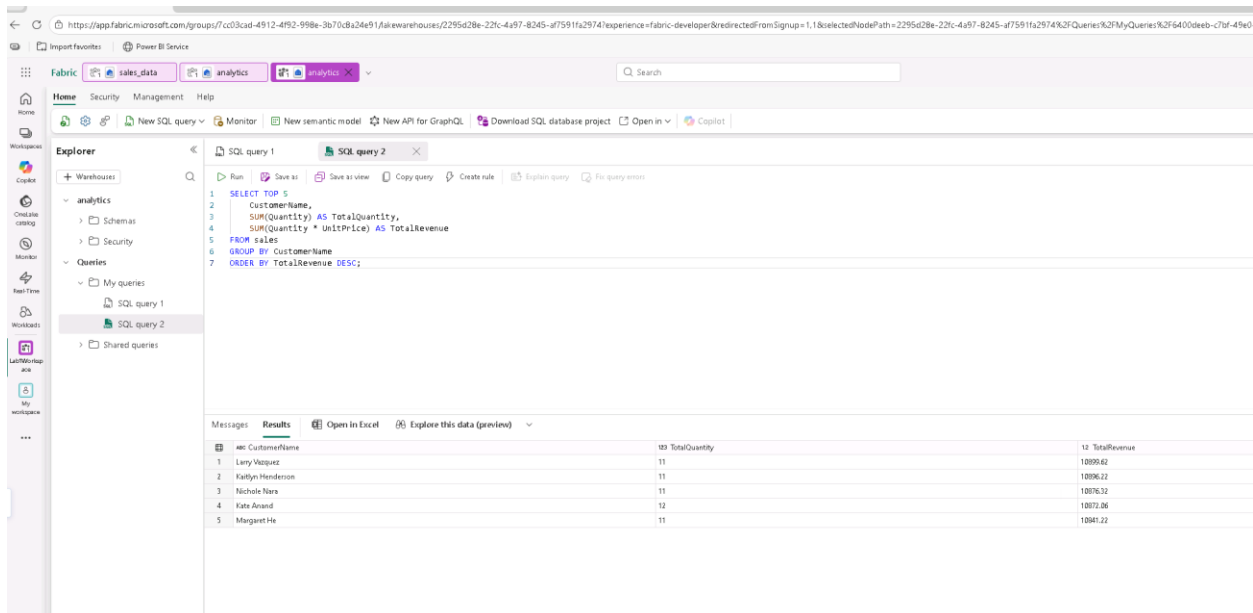
```

SELECT TOP 5
    CustomerName,
    SUM(Quantity) AS TotalQuantity,
    SUM(Quantity * UnitPrice) AS TotalRevenue
FROM sales
GROUP BY CustomerName
ORDER BY TotalRevenue DESC;

```

- Run the query to identify the top five customers by total revenue.

Figure 12 — Querying top customers by revenue using the SQL endpoint



This image shows the SQL query and results table listing the top five customers ranked by total revenue. The output demonstrates how the SQL endpoint supports ad-hoc analysis and ranking queries without requiring data export or external tools.

Step-by-Step Breakdown

- **FROM sales:** The database grabs the raw transaction data from the sales table.
- **GROUP BY CustomerName:** The query groups individual transactions together so calculations are performed for each unique buyer.
- **SUM(Quantity) AS TotalQuantity:** It adds up every item the customer ever bought and labels it **TotalQuantity**.
- **SUM(Quantity * UnitPrice) AS TotalRevenue:** It calculates the total dollar amount the customer spent across all purchases and labels it **TotalRevenue**.
- **SELECT TOP 5 CustomerName, ...:** It outputs the customer's name alongside their calculated totals, but strictly limits the final list to the first **5** records.
- **ORDER BY TotalRevenue DESC:** It sorts the customers by their total spend in descending order so the biggest spenders land at the very top (allowing the **TOP 5** filter to grab the correct people).

Structure:

- The SQL endpoint exposes Delta Lake tables through a standard SQL interface.
- Queries execute directly against OneLake storage, using the same data as notebooks and semantic models.
- Results can be previewed, exported, or used to build downstream reports.
- The endpoint supports full SQL syntax, including aggregation, filtering, and ordering.

Function:

- Enables real-time analytics without data movement or duplication.
- Provides a familiar SQL interface for querying Lakehouse data.
- Ensures consistency between engineering and analytical views of the same dataset.
- Simplifies validation and exploration of data before modeling or visualization.
- Forms the foundation for semantic model creation and Power BI integration.

Summary:

The SQL endpoint enables fast, reliable analysis of Delta Lake data directly within the Lakehouse.

With the SQL endpoint validated and analytical queries confirmed that the next step is to transform these raw analytical results into a governed, reusable business layer. This is achieved by creating a **semantic model**, which defines standardized measures, relationships, and metadata that reporting tools such as Power BI can use for consistent enterprise analytics.

8. Create and Explore Semantic Model

Purpose: Build a governed, business-friendly model that exposes Lakehouse data to reporting tools such as Power BI, enabling consistent metrics, relationships, and analytical logic across the organization.

A semantic model provides a structured layer on top of Delta Lake tables. It defines measures, relationships, metadata, and business logic that reporting tools rely on. Unlike raw tables, semantic models present fields in a user-friendly format and ensure that all users work from the same governed definitions.

Creating a semantic model in Fabric demonstrates how Lakehouse data becomes consumable for enterprise reporting.

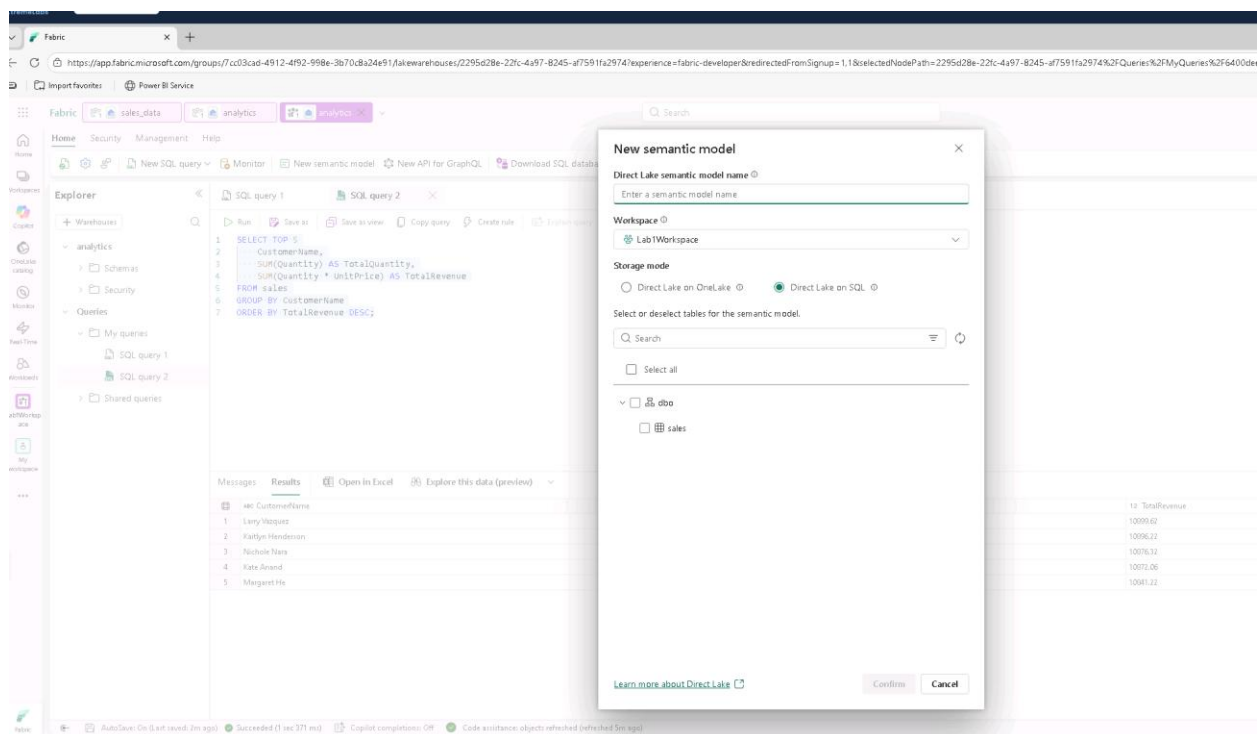
8.1 Create a New Semantic Model

Steps completed:

- Open the **analytics** Lakehouse
- Select **New semantic model**
- Choose **Direct Lake on SQL**
- Select the **sales** table
- Enter the model name (**sales_model**)
- Select **Confirm**

Fabric generates a semantic model that reads the Delta Lake table directly through Direct Lake mode. This allows the model to access the underlying data without import or duplication.

Figure 13 — Creating a semantic model from the sales table



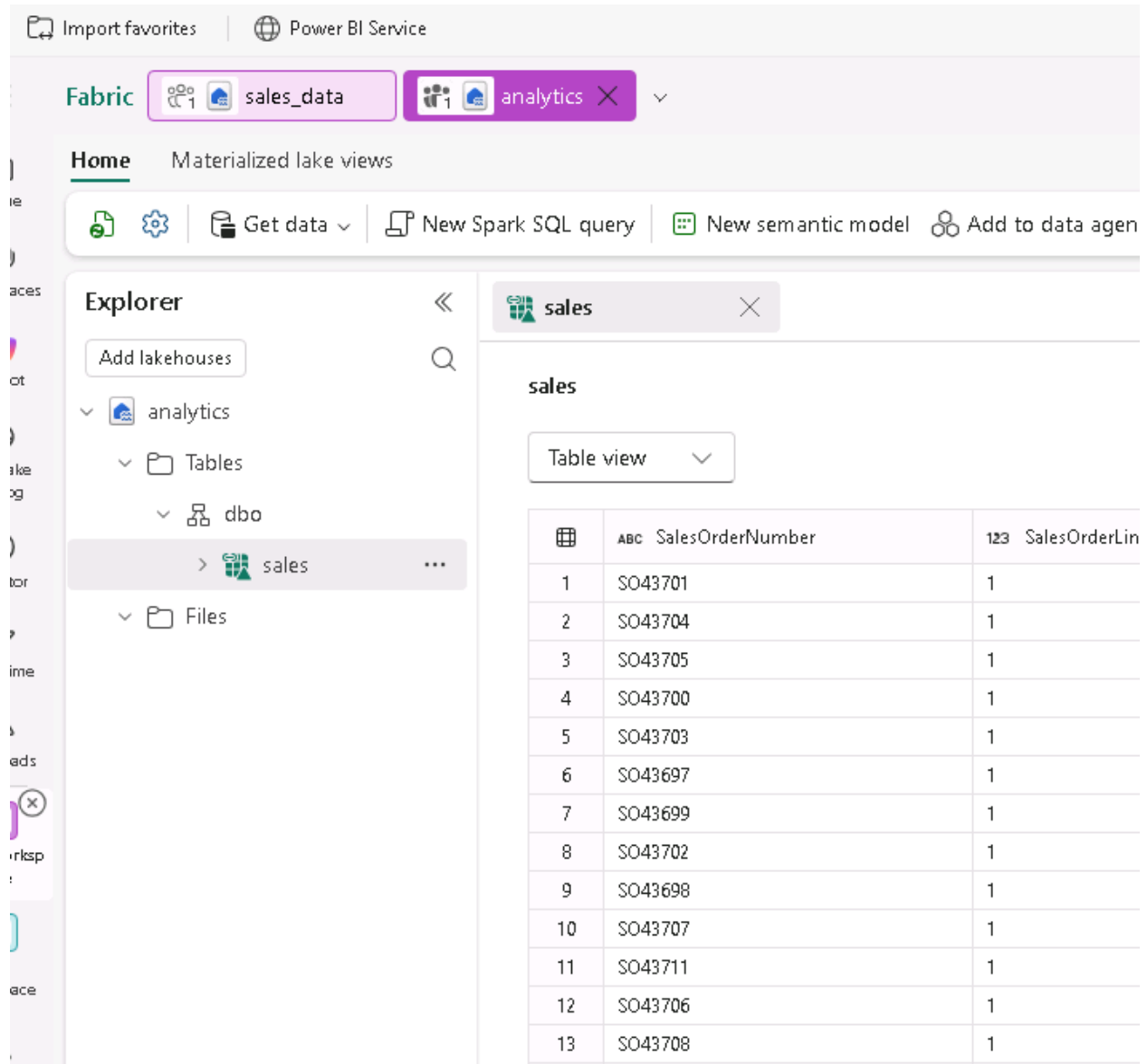
This image shows the semantic model creation dialog with the **sales** table selected and Direct Lake mode enabled

8.2 Explore Model Fields

Steps completed:

- Open **sales_model**
- Expand **Fields**
- Review automatically generated fields

Figure 14 — Viewing fields in the semantic model



The screenshot displays the Microsoft Fabric interface. At the top, there's a navigation bar with 'Import favorites' and 'Power BI Service'. Below this, the 'Fabric' logo is visible, followed by tabs for 'sales_data' and 'analytics'. The 'Home' section is titled 'Materialized lake views'. A toolbar contains icons for 'Get data', 'New Spark SQL query', 'New semantic model', and 'Add to data agent'. The 'Explorer' pane on the left shows a tree structure: 'analytics' (expanded) > 'Tables' (expanded) > 'dbo' (expanded) > 'sales' (selected). The main pane shows the 'sales' table in 'Table view' mode. The table has two columns: 'SalesOrderNumber' and 'SalesOrderLine'. The data is as follows:

	ABC SalesOrderNumber	123 SalesOrderLine
1	SO43701	1
2	SO43704	1
3	SO43705	1
4	SO43700	1
5	SO43703	1
6	SO43697	1
7	SO43699	1
8	SO43702	1
9	SO43698	1
10	SO43707	1
11	SO43711	1
12	SO43706	1
13	SO43708	1

This image shows the **sales** table opened in the analytics Lakehouse. The table view confirms that the Delta Lake data is accessible and structured correctly, ready for use in the semantic model and downstream reporting.

8.3 Add Basic Measures

Steps completed:

- Open **Model** view
- Select **New measure**

Create the following measures:

Total Revenue

SUM(sales[Quantity] * sales[UnitPrice])

Total Quantity

SUM(sales[Quantity])

These measures define standardized business logic that will be reused across all downstream reports. By embedding calculations such as revenue and quantity directly into the semantic model, Fabric ensures that every report uses the same consistent definitions without requiring users to write SQL.

Structure:

- A semantic model defines business-friendly fields, measures, and metadata.
- It provides a governed layer on top of Delta Lake tables.
- Direct Lake mode allows the model to read Delta Lake files directly without import.
- Measures store reusable analytical logic such as revenue, quantity, or margins.
- Relationships between tables can be defined to support multi-table reporting without requiring SQL JOINS.
- The model serves as the foundation for Power BI reports and enterprise analytics.

Function:

- A semantic model turns complex, raw data tables into a user-friendly layer of standardized business metrics and relationships.
- It eliminates the need for users to write manual SQL by pre-defining logic such as **Total Revenue**, handling table connections, and securing data access.
- This ensures that everyone across the organization builds reports using the exact same, accurate calculations.
- A new semantic model is necessary when raw, granular Delta Lake data must be translated into business-friendly concepts that users can easily understand and query without writing SQL.

Bridging the Gap Between Raw Data and Users:

- **Translating technical names:** Raw tables often use technical column names (e.g., *CustomerName*, *UnitPrice*). A semantic model can expose cleaner, business-friendly names (e.g., *Client*, *Retail Price*).
- **Pre-defining complex logic:** Instead of requiring users to manually write **SUM(Quantity * UnitPrice)** to calculate revenue, the semantic model embeds this logic into a reusable metric such as **Total Revenue**, ensuring consistent calculations across all reports.
- **Establishing relationships:** It defines how tables connect (e.g., linking a sales table to a customer dimension), allowing users to drag and drop fields into a report without understanding SQL JOINS.
- **Optimizing performance and security:** Semantic models can pre-aggregate large datasets for faster loading in tools like Power BI. They also enforce Row-Level Security (RLS), ensuring users only see data they are authorized to view (e.g., regional managers only seeing their territory's sales).

Summary:

The semantic model exposes Lakehouse data through a governed, business-friendly interface. By defining measures, relationships, and metadata, it ensures that all users work from the same authoritative logic, enabling consistent and reliable reporting across the organization.

With the semantic model created and enriched with standardized business logic, the next step is to explore how it supports downstream analytics and reporting workflows in Power BI.

9. Use Semantic Model in Power BI

Purpose: Demonstrate how the semantic model enables interactive reporting and visualization using standardized business logic. This step confirms that the model's measures and relationships can be used directly in Power BI or Fabric's analytics preview without manual SQL.

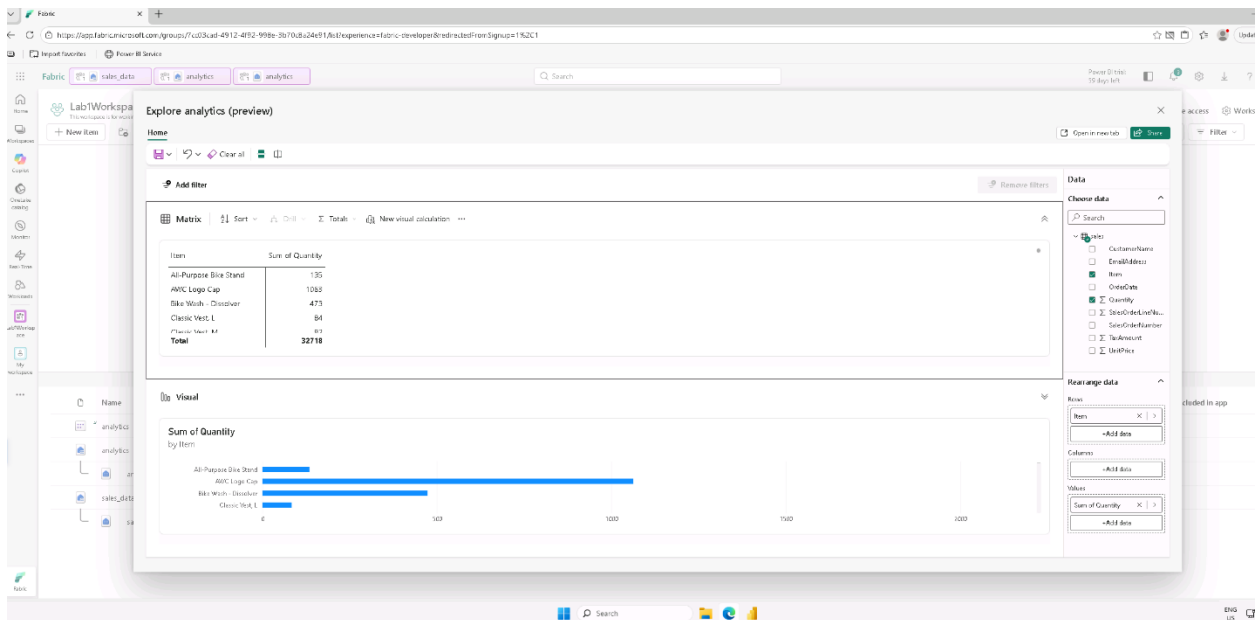
Power BI and Fabric's integrated analytics tools connect directly to semantic models, allowing users to build dashboards and visuals that automatically apply governed calculations such as **Total Revenue** and **Total Quantity**. This ensures consistent reporting across all users and departments.

9.1 Explore Analytics Preview

Steps completed:

Open the **sales_model** semantic model Select **Explore analytics (preview)** Choose **Matrix** and **Visual** views Drag **Item** to the Rows field Drag **Quantity** to the Values field Fabric automatically aggregates the data and displays both a matrix summary and a bar chart visualization.

Figure 16 — Exploring semantic model analytics preview



This image shows the analytics preview interface displaying a matrix and bar chart summarizing item quantities. The visualization confirms that the semantic model fields are accessible and that calculations execute correctly without manual SQL.

Structure:

- Power BI and Fabric analytics connect directly to semantic models.
- Measures and metadata defined in the model are automatically available for visualization.
- Visuals use the semantic model's logic rather than raw SQL queries.
- Direct Lake mode enables fast, interactive reporting on large datasets.

Function:

- Provides a governed reporting layer that ensures consistent calculations across dashboards.
- Allows users to build visuals without writing SQL or managing table joins.
- Ensures that business logic such as **Total Quantity** and **Total Revenue** is applied uniformly.
- Enables interactive exploration of Lakehouse data through Fabric's analytics preview or Power BI.
- Supports enterprise-grade performance and security.

Summary:

The semantic model integrates seamlessly with Power BI and Fabric's analytics preview, enabling users to visualize and analyze data interactively. By using predefined measures and relationships, reports remain consistent, accurate, and aligned with governed business logic.

10. Use Copilot to Generate SQL Queries (Feature Not Available in This Environment)

Purpose: Copilot in Fabric provides natural-language assistance for writing SQL queries, enabling faster exploration of warehouse data. When available, Copilot can interpret user intent and generate complete SQL statements without requiring manual coding.

The lab environment used for this exercise does not currently include access to Copilot inside the SQL analytics endpoint. As a result, the Copilot-assisted SQL generation task cannot be completed at this time. When access to a version of Fabric that supports Copilot in the SQL editor becomes available, this step will be added to the repository.

Importance of Copilot for SQL Assistance

Structure:

- Copilot appears directly in the SQL query editor toolbar.
- It accepts natural-language prompts such as *"Show total revenue by month for 2026."*

- It generates full SQL statements using the warehouse's schema and metadata.
- Users can insert, modify, and run the generated queries immediately.

Function:

- Copilot accelerates SQL development by reducing manual query writing.
- It helps users explore data even if they are not fluent in SQL syntax.
- It ensures consistent query patterns across users and teams.
- It supports iterative analysis by allowing rapid generation of alternative queries.
- Once available, Copilot will provide a governed, AI-assisted workflow for warehouse analytics.

Summary:

Copilot is an important feature for enhancing SQL productivity in Fabric. Although it is not available in the current lab environment, its role in generating SQL queries, supporting natural-language exploration, and improving analytical workflows is essential for scalable data analysis. When Copilot access becomes available, the SQL-generation tasks will be completed and added to this lab's documentation.

11. Clean Up Resources

In this exercise, you created a Lakehouse, explored the OneLake catalog, created shortcuts for cross-workspace access, queried data using the SQL analytics endpoint, and built a semantic model. After completing these tasks, the temporary workspaces used for the lab can be safely removed to prevent unnecessary capacity usage.

Steps completed:

- Open the workspace used for the lab
- Select **Workspace settings** from the toolbar
- In the **General** section, choose **Remove this workspace**
- Confirm deletion when prompted
- Repeat these steps for any additional workspaces created during the exercise (such as the analytics workspace)

Deleting the workspaces ensures that no unneeded items remain in the environment once the lab is complete.

12. Reflection

This lab demonstrated how Microsoft Fabric supports end-to-end analytical workflows using Lakehouses, shortcuts, SQL endpoints, and semantic models. The sequence of tasks showed how raw data stored in OneLake can be transformed into structured, governed analytical assets that support consistent reporting across an organization.

The process began by creating dedicated workspaces and Lakehouses to store and manage data. Exploring the OneLake catalog highlighted how Fabric centralizes data discovery and metadata, enabling users to locate assets quickly. Creating shortcuts reinforced Fabric's single-copy architecture by allowing cross-workspace access without duplicating data.

Querying the Delta Lake tables through the SQL analytics endpoint illustrated how Fabric exposes Lakehouse data through a relational interface. Writing aggregation queries demonstrated how structured tables, schema design, and SQL logic produce meaningful business insights from raw transactional data. Building a semantic model further emphasized the importance of governed metrics, relationships, and business-friendly metadata. The semantic model ensured that calculations such as Total Revenue and Total Quantity behaved consistently across tools and users.

Although Copilot assistance was not available in the current environment, its role in accelerating SQL development and supporting natural-language exploration remains an important part of the broader Fabric ecosystem. When access becomes available, Copilot-generated SQL tasks will be added to complete the analytical workflow.

Overall, this lab reinforced how Fabric integrates storage, SQL analytics, semantic modeling, and catalog governance into a unified platform. The completed steps demonstrated how each component contributes to reliable, scalable analytics, forming a foundation for enterprise reporting and future data modeling work.